

Reviewer Pack — Coxeter Group Action Complexity of SAT

Entry be7ddd28a550 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 12:24:22 UTC

Coxeter Group Action Complexity of SAT

Entry ID: be7ddd28a550 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 12:24:22 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Coxeter Groups) **Field B** (complexity object): Boolean Function Complexity: Resolution Proof Complexity

Statement:

The resolution proof width of a k -CNF instance is upper-bounded by the order of the largest element in its associated Coxeter group action, i.e., $E[|W_n(k)|] \leq O(n^{(\alpha+1)})$, where $W_n(k)$ represents the Coxeter group action on an n -vertex graph with k edges.

Rationale (proposer's reasoning):

Coxeter groups provide a combinatorial description of symmetry, and their actions can be used to capture the complexity of computations. If the action complexity is high, it implies that the resolution proof width should also be large, potentially providing a new perspective on NP-completeness.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6806f0eb840e1d7f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the empirical correlation coefficient between the resolution proof width and the order of the largest element in the Coxeter group action for at least 80% of the seeds is ≥ 0.5 , with no seed showing a correlation coefficient ≤ 0.2 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter Group action" AND "resolution proof complexity" - "k-CNF instance" AND "order of largest element" IN "Coxeter group" - "Boolean function complexity" AND "SAT problem" AND "Coxeter groups"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_k_cnf(n, k):
    variables = list(range(1, n + 1))
    clauses = set()
    while len(clauses) < k:
        clause = {random.choice(variables), random.choice(variables)}
        if clause not in clauses and len(clause) == 2:
            clauses.add(tuple(sorted(clause)))
    return clauses

def coxeter_group_action(graph):
    n = len(graph)
    elements = [{i} for i in range(n)]
    for v, neighbors in graph.items():
        new_elements = set()
        for e in elements:
            if v not in e:
                new_e = e.union(neighbors)
                new_elements.add(tuple(sorted(new_e)))
        elements = new_elements
    return max(len(e) for e in elements)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    n_max = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
```

```

    for _ in range(5):
        k = random.randint(1, min(n * (n - 1) // 2, 10))
        graph = {i: set() for i in range(n)}
        for u, v in generate_k_cnf(n, k):
            graph[u].add(v)
            graph[v].add(u)
        resolution_width = len(graph) + k
        order_largest_element = coxeter_group_action(graph)
        total_metric_value += resolution_width * order_largest_element
        instances_tested += 1
        n_max = max(n_max, n)

mean_metric_value = total_metric_value / instances_tested
support_fraction = Fraction(instances_tested, len(n_values) * 5).limit_denominator()

if support_fraction < Fraction(4, 5):
    conjecture_holds = False
    counterexample = "support_fraction_too_low"

return {
    "metric_name": "resolution_width * order_largest_element",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = Fraction(sum(1 for r in results if r["conjecture_holds"]), len(results)).limit_denominator()

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(r["counterexample"] == "support_fraction_too_low" for r in results):
    first_failing_seed = next(r["seed"] for r in results if r["counterexample"] == "support_fraction_too_low")
    print(f"RESULT: FALSIFIED counterexample=\"support_fraction_too_low\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE support_fraction_too_low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f82fe4b0.py", line 82, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f82fe4b0.py", line 54, in run_trial
    graph[v].add(u)
    ~~~~~^^^
```

KeyError: 5

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that it was not possible to evaluate the empirical correlation coefficient as required by the support condition. | next: Re-run the test without errors to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14683
2	preregistration	ollama_remote	glm4:latest	0	0	9306
3	novelty	ollama_remote	glm4:latest	0	0	8433
4	novelty	ollama_remote	glm4:latest	0	0	26056
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17208
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	53033
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13385
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12277
9	judge	ollama_remote	glm4:latest	0	0	20197

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 174578 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/be7ddd28a550.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/be7ddd28a550.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/be7ddd28a550.tar.gz (if generated)